

PEAR: Progressive Evidence-Based AutoResearch for Industrial Search Systems

Global E-Commerce Agentic Search Team

Abstract

AutoResearch improves systems through iterative experimentation: agents propose candidate modifications, evaluate them, and use the results to guide subsequent exploration. Applying this paradigm to industrial search presents two challenges. (1) Common AutoResearch approaches follow a keep-if-better rule, retaining the highest-scoring candidate for subsequent experiments. Under non-stationary traffic, transient gains may be mistaken for persistent improvements, impairing reliable accumulation of search knowledge. (2) Candidate modifications can be evaluated at multiple fidelity levels, from low-cost proxies to online validation, differing in cost, objective alignment, and statistical reliability. Existing methods rely on individual signals or task-specific procedures, lacking a unified basis for using evidence across levels to guide search. We introduce **Progressive Evidence-Based AutoResearch (PEAR)** with two complementary components. Evidence-driven AutoResearch maintains an independent, hypothesis-guided research state for each strategy task within a predefined objective and intervention scope. Each state evolves through a Plan–Execute–Evaluate–Update transition that links experimentation to context-aware evidence interpretation and hypothesis revision. Confidence-Gated Verifier Ladder organizes evaluation into four levels of increasing fidelity: Offline Replay, Shadow-Traffic Evaluation, Rapid Online Evaluation, and Decision-Grade Online Evaluation. A unified confidence-based gate promotes candidates only when evidence supports a statistically significant positive effect, enabling broad low-cost exploration while reserving costly online experiments for promoted candidates. In a real-world industrial search system, strategies optimized with PEAR significantly increased Main Order/DAU by 2.7336% and 3.2957% relative to their respective baselines in two A/B experiments.

1 Introduction

Industrial search systems integrate query understanding, candidate retrieval, and result ranking to determine which results are presented to users. Recent work has substantially advanced individual components through instruction-aware retrieval and LLM-based reranking [3, 14, 26, 27]. In production, however, these components operate within a multi-stage decision process in which models, ranking strategies, and business constraints interact. Shifts in user demand, content supply, and business objectives require these strategies to be continually revisited, while dependencies across stages make the effects of candidate modifications context-dependent and difficult to predict offline. Engineers therefore improve the system through iterative experimentation: they formulate hypotheses from observed system behavior, implement candidate strategies, evaluate them under real traffic, and use the results to guide subsequent iterations.

Recent advances in large language model (LLM) agents have demonstrated multi-step reasoning, tool use, and the ability to act in external environments [17, 18, 20, 23–25], creating an opportunity to automate

portions of this workflow. AutoResearch turns these capabilities into an iterative experimentation loop in which agents propose and implement candidate modifications, evaluate their outcomes, and use the resulting evidence to guide subsequent proposals [1, 5, 15, 19]. Recent work has applied this paradigm to search and recommendation systems, exploring interventions that range from ranking parameters to model architectures and reward functions [4, 22], and evaluating them using offline proxy metrics, model diagnostics, simulated user responses, and online A/B test outcomes [7, 9]. Collectively, these studies demonstrate the potential of agent-driven strategy optimization, while making experimental evidence the primary basis for candidate selection and subsequent exploration.

Reliably using evidence from both offline and online evaluations in AutoResearch for industrial search presents two challenges arising from non-stationary outcomes and heterogeneous evaluation procedures. **(1) How can an agent accumulate reliable experimental knowledge when candidate effects are context-dependent?** Many AutoResearch systems apply a keep-if-better rule, retaining the highest-scoring candidate as the reference for subsequent experiments [15, 19]. This rule implicitly assumes that evaluation scores remain comparable across rounds. In industrial search, traffic composition and system state evolve over time; consequently, experiments conducted under different conditions may estimate candidate effects that are not directly comparable. A transient or context-specific gain may therefore be recorded as a persistent improvement, weakening hypothesis assessment and misdirecting subsequent exploration. **(2) How should evidence from different evaluation procedures govern candidate promotion?** Low-cost proxy evaluations support broad exploration, but their improvements may not translate into online gains. Online experiments provide evidence that is more directly aligned with deployment objectives, but sufficiently evaluating every candidate online is costly and risky [6, 8, 11, 12, 16]. Existing approaches use individual evaluation signals or task-specific evaluation procedures [4, 7, 9, 22], leaving unclear how evidence from different procedures should jointly inform promotion decisions. The resulting challenge is to accumulate reliable experimental knowledge under changing contexts and make sound candidate-promotion decisions from heterogeneous evaluation evidence.

To address these challenges, we introduce **Progressive Evidence-Based AutoResearch (PEAR)**, a framework for industrial search systems that couples persistent strategy states with progressive candidate evaluation through two complementary components. **Evidence-driven AutoResearch** maintains an independent research state for each strategy task, comprising the experimental context, current hypotheses, candidate intervention space, and accumulated evidence. The state evolves through a Plan–Execute–Evaluate–Update transition augmented with Bayesian-optimization-inspired reasoning. This reasoning balances refinement of evidence-supported regions with exploration of underexplored regions, while experimental results are interpreted within their contexts to support hypothesis revision and knowledge accumulation across experiments. **Confidence-Gated Verifier Ladder** organizes candidate evaluation into four progressively higher-fidelity levels through a unified confidence-based promotion gate: Offline Replay, Shadow-Traffic Evaluation, Rapid Online Evaluation, and Decision-Grade Online Evaluation. Evaluation results and promotion decisions from each level update the corresponding research state and inform subsequent search. Lower-cost evaluations support broad exploration, while costly online experiments are reserved for promoted strategies. In a real-world industrial search system, strategies optimized with PEAR significantly increased Main Order/DAU by 2.7336% and 3.2957% relative to their respective baselines in two A/B experiments at the Decision-Grade Online Evaluation stage.

2 Contributions

1. We introduce **Evidence-driven AutoResearch** to support reliable knowledge accumulation for industrial search optimization under non-stationary traffic. For each strategy task, it maintains an independent research state that links hypotheses and candidate interventions to their evaluation outcomes and experimental contexts. The state evolves through a Plan–Execute–Evaluate–Update transition, in which contextual evidence informs revisions to both the hypotheses and the candidate intervention space across

experimental rounds.

2. We develop **Confidence-Gated Verifier Ladder**, which organizes candidate evaluation into four levels of progressively greater fidelity and cost, from Offline Replay to Decision-Grade Online Evaluation. The ladder preserves each level’s evaluation signal in a unified evidence record and applies a common confidence-based promotion rule, allowing low-cost evaluations to support broad exploration while reserving costly online experiments for statistically supported candidates. The resulting evidence and promotion decisions are returned to the research state to guide subsequent search.
3. We deploy and evaluate PEAR in a real-world industrial search system using three strategy tasks. The evaluation documents evidence-driven hypothesis revision and, for two promoted candidates, directionally consistent positive effects from Shadow-Traffic Evaluation to Decision-Grade Online Evaluation. In two Decision-Grade Online A/B experiments, strategies optimized with PEAR significantly increased Main Order/DAU by 2.7336% and 3.2957% relative to their respective baselines.

3 Related Work

3.1 AutoResearch

AutoResearch organizes candidate generation, experimentation, and evaluation into iterative research workflows [5, 15, 21]. A common update mechanism is the keep-if-better rule, which retains candidates that improve the observed experimental score as references for subsequent search [15, 19]. The interpretation of these improvements depends on the comparability of evaluation outcomes across experiments. Recent systems preserve experimental history to inform subsequent proposals. Meta-Harness optimizes harness code using the code, execution traces, and scores of prior candidates [10]. AiScientist maintains a persistent workspace that retains experimental artifacts across implementation, experimentation, and diagnosis [2]. AutoResearchClaw combines multi-agent deliberation, execution recovery, and cross-run experience accumulation to support iterative hypothesis generation and experimentation [13]. These approaches provide mechanisms for retaining and reusing information across experiments.

The keep-if-better rule uses observed candidate scores to guide subsequent search. In industrial search optimization, non-stationary traffic complicates the interpretation of these scores across experimental contexts. Reliable search knowledge accumulation requires distinguishing persistent improvements from transient gains over multiple experiments.

3.2 AutoResearch in Search and Recommendation

Recent studies apply AutoResearch to search and recommendation through task-specific candidate generation and evaluation procedures. Self-Evolving Recommendation System combines a fast offline loop for generating and filtering model candidates with a slower online loop for validating delayed business outcomes [22]. Self-EvolveRec combines semantic feedback from a user simulator with model-diagnostic signals to guide candidate modifications across the recommendation pipeline [7]. These studies use proxy evaluation and structured feedback to guide subsequent candidate generation.

Other systems incorporate online experimental feedback into iterative optimization. Sortify estimates the relationship between offline signals and online outcomes and maintains cross-round memory to inform subsequent parameter search [4]. AgentX connects candidate generation, implementation, online experimental evaluation, and experience feedback through specialized agents [9]. These systems demonstrate how online outcomes can inform subsequent search in deployed search and recommendation systems.

Existing approaches organize candidate evaluation around individual signals or task-specific procedures. Proxy and online evaluation results differ in acquisition cost, objective alignment, and statistical reliability. A unified

basis for using these results to guide candidate selection and subsequent search remains an open problem.

4 Problem Setup

AutoResearch for industrial search optimization iteratively refines search strategies through candidate generation, experimentation, and evaluation. Each strategy task k specifies a predefined objective and intervention scope. A candidate strategy c specifies a testable modification to the search system, such as a parameter configuration or a change to the ranking mechanism. Multiple strategy tasks may proceed concurrently within their respective intervention scopes.

At experimental round t , the agent generates a candidate set $\mathcal{A}_{k,t}$ for strategy task k . An evaluation procedure assigns scores to the candidates according to the optimization objective. The agent uses the resulting scores to refine subsequent proposals. Repeated candidate generation, evaluation, and proposal revision form an iterative optimization process.

In AutoResearch for industrial search systems, evaluations range from low-cost proxy evaluation to online validation. These evaluation sources differ in acquisition cost, objective alignment, and statistical reliability. Proxy evaluation supports broad candidate exploration at low cost. Online validation measures candidate effects through real-user outcomes and requires additional traffic and observation time. Let $\mathcal{S}(c)$ denote the set of evaluation stages applied to candidate c , and let $\mathcal{E}_\ell(c)$ denote the evidence acquired at stage ℓ . The collected evidence is

$$\mathcal{E}(c) = \{\mathcal{E}_\ell(c) \mid \ell \in \mathcal{S}(c)\}. \quad (1)$$

For strategy task k , $\mathcal{E}_{k,t}^{(\ell)}$ denotes the set of stage- ℓ evidence records acquired in round t . Each result reflects its evaluation protocol and traffic context. Under non-stationary traffic, results from different rounds must be interpreted within their corresponding experimental contexts.

The objective is to optimize industrial search strategies through iterative experimentation guided by heterogeneous evaluation results. Evaluation results provide the basis for hypothesis revision, candidate selection, and subsequent search. This process requires reliable search knowledge accumulation across experimental rounds and sufficient online validation to support deployment decisions.

5 Progressive Evidence-Based AutoResearch (PEAR)

As illustrated in Figure 1, PEAR couples persistent strategy states with progressive candidate evaluation through two complementary components: Evidence-driven AutoResearch and Confidence-Gated Verifier Ladder.

Evidence-driven AutoResearch maintains an independent, hypothesis-guided research state for each strategy task. Each state preserves the hypotheses, candidate history, and experimental contexts associated with that task. A Plan–Execute–Evaluate–Update transition connects candidate generation and experimentation with evidence interpretation and hypothesis revision.

Confidence-Gated Verifier Ladder organizes candidate evaluation into four progressively higher-fidelity levels: Offline Replay, Shadow-Traffic Evaluation, Rapid Online Evaluation, and Decision-Grade Online Evaluation. Unified evidence records and a confidence-based promotion gate connect these evaluation levels. Evaluation results and promotion decisions update the corresponding research states and inform subsequent search. Lower-cost evaluations support broad exploration, and costly online experiments are reserved for promoted candidates.

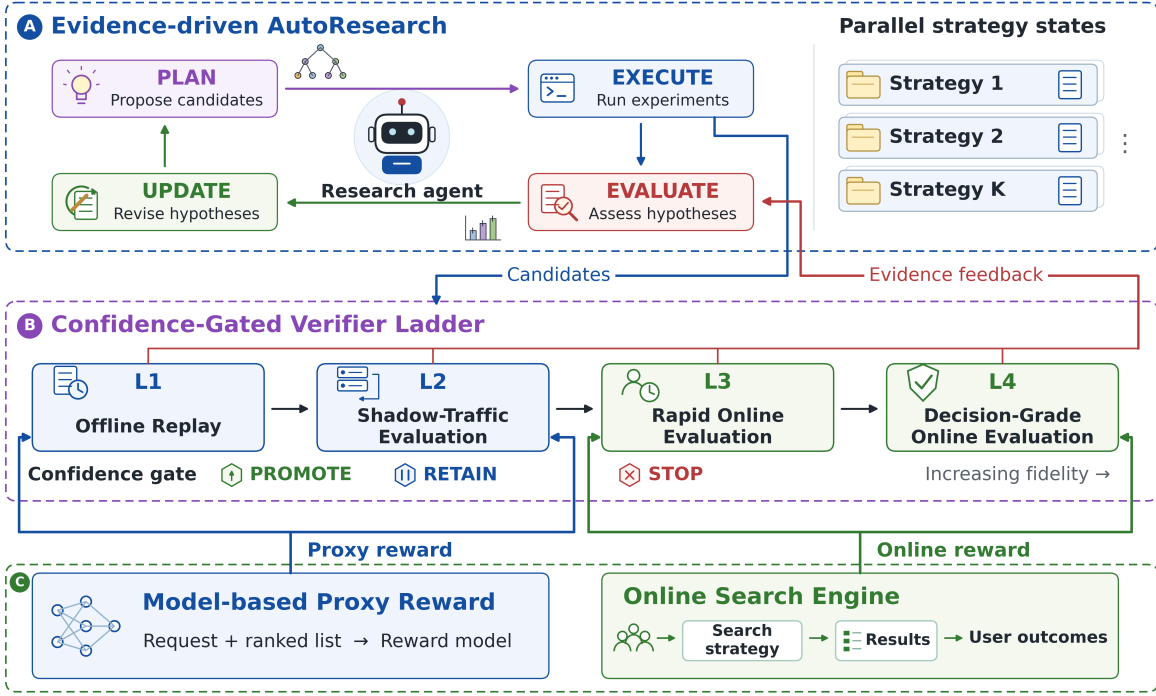


Figure 1 Overview of PEAR. Evidence-driven AutoResearch maintains parallel strategy-level research states, while the Confidence-Gated Verifier Ladder progressively evaluates candidates, governs their promotion using evidence of increasing fidelity, and returns evaluation results to the corresponding research states.

5.1 Evidence-driven AutoResearch

Evidence-driven AutoResearch associates optimization hypotheses with the evaluation evidence acquired for each strategy task. A hypothesis describes the expected effect of a candidate modification under specified conditions. Its associated evidence records the candidate, evaluation outcome, and experimental context. This association provides the basis for interpreting non-stationary evaluation results and revising hypotheses across experiments.

For strategy task k at experimental round t , the research state is

$$\mathcal{H}_{k,t} = (C_k, \mathcal{M}_{k,t}, \Omega_{k,t}, \mathcal{D}_{k,t}), \quad (2)$$

where C_k specifies the search-system implementation, optimization objective, intervention scope, and experimental protocol. $\mathcal{M}_{k,t}$ contains the current hypotheses, $\Omega_{k,t}$ denotes the candidate intervention space within the specified scope, and $\mathcal{D}_{k,t}$ contains the accumulated evidence and its associations with the hypotheses. Independent states preserve the experimental context of each strategy task and support concurrent exploration across strategy tasks.

The research state evolves through a Plan–Execute–Evaluate–Update transition:

$$\mathcal{H}_{k,t} \xrightarrow{\text{PLAN}} \mathcal{A}_{k,t} \xrightarrow{\text{EXECUTE}} \Delta\mathcal{D}_{k,t} \xrightarrow{\text{EVALUATE}} \mathcal{R}_{k,t} \xrightarrow{\text{UPDATE}} \mathcal{H}_{k,t+1}. \quad (3)$$

Here, $\mathcal{A}_{k,t}$ is the candidate set, $\Delta\mathcal{D}_{k,t}$ is the newly acquired evidence, and $\mathcal{R}_{k,t}$ records the evidence-based assessment of the current hypotheses.

PLAN: Hypothesis-Guided Candidate Generation. The agent formulates testable hypotheses from the mechanisms of the industrial search system, the optimization objective, and accumulated experimental

evidence. These hypotheses describe how candidate modifications are expected to affect the objective under the corresponding experimental conditions. Each candidate in $\mathcal{A}_{k,t}$ is associated with a hypothesis and an expected observation. Candidate generation includes interventions that examine individual effects, test interactions, or replicate previous observations.

Repeated refinement around previously promising candidates can concentrate exploration within a narrow region. To broaden candidate generation, the agent incorporates Bayesian-optimization-inspired reasoning:

$$\mathcal{A}_{k,t} = \mathcal{G}(C_k, \mathcal{M}_{k,t}, \mathcal{D}_{k,t}, \Omega_{k,t}; \mathcal{K}_{\text{BO}}), \quad \mathcal{A}_{k,t} \subseteq \Omega_{k,t}, \quad (4)$$

where $\mathcal{G}$ denotes the agent’s candidate-generation procedure and $\mathcal{K}_{\text{BO}}$ denotes Bayesian-optimization-inspired reasoning principles. Conditioned on the search-system context, current hypotheses, and accumulated evidence, this procedure combines refinement of evidence-supported regions with exploration of underexplored regions. These principles guide qualitative reasoning about observed effects and uncertainty rather than specifying a fitted numerical optimization model.

EXECUTE: Experiment Execution and Evidence Collection. The agent invokes stage-specific experimentation tools to apply candidate modifications within the corresponding evaluation environments and initiate experiments under Confidence-Gated Verifier Ladder. As candidates progress through the evaluation stages, the agent retrieves the evaluation results and promotion decisions produced by the corresponding verifiers. These outputs form the newly acquired evidence $\Delta\mathcal{D}_{k,t}$, with each record linked to its candidate, hypothesis, and experimental context.

EVALUATE: Contextual Hypothesis Assessment. The agent assesses each hypothesis using its associated evidence in $\mathcal{D}_{k,t} \cup \Delta\mathcal{D}_{k,t}$. This assessment considers the candidate modifications, evaluation stages, and experimental contexts underlying the observed effects. Consistent observations provide support for a hypothesis. Conflicting observations motivate examination of its assumptions and applicability conditions. Inconclusive observations remain unresolved evidence for further experimentation. The assessment record $\mathcal{R}_{k,t}$ preserves these distinctions for hypothesis revision.

UPDATE: Evidence-Based Hypothesis Revision. The assessment record updates the hypotheses and the candidate intervention space:

$$\begin{aligned} (\mathcal{M}_{k,t+1}, \Omega_{k,t+1}) &= \mathcal{U}(\mathcal{M}_{k,t}, \Omega_{k,t}, \mathcal{R}_{k,t}), \\ \mathcal{D}_{k,t+1} &= \mathcal{D}_{k,t} \cup \Delta\mathcal{D}_{k,t}, \end{aligned} \quad (5)$$

where $\mathcal{U}$ denotes evidence-based revision of the hypotheses and candidate intervention space. Supported hypotheses guide further refinement and replication. Conflicting evidence motivates revised hypotheses and alternative candidate modifications within the specified intervention scope. Evaluation results and promotion decisions remain available in the updated state, allowing subsequent proposals to build on accumulated evidence and its experimental context.

5.2 Confidence-Gated Verifier Ladder

Confidence-Gated Verifier Ladder organizes heterogeneous candidate evaluations into four levels: Offline Replay (L1), Shadow-Traffic Evaluation (L2), Rapid Online Evaluation (L3), and Decision-Grade Online Evaluation (L4). **All four levels are aligned with the same optimization objective but differ in signal source, evaluation fidelity, acquisition cost, traffic exposure, and observation horizon.** L1 and L2 use a model-based proxy score that estimates this objective from ranking outputs, and L3 and L4 use the objective

directly observed from online user outcomes. Each strategy task instantiates an ordered sequence of active levels according to data availability and validation requirements. A unified confidence-based promotion rule connects consecutive levels in this task-specific sequence. Each active level produces evaluation results under its corresponding experimental protocol. Evaluation results and promotion decisions provide evidence for hypothesis assessment and subsequent search.

5.2.1 Unified Promotion Policy

The promotion policy uses the estimated candidate effect and its confidence interval at each evaluation stage. For candidate c at stage ℓ , the verifier produces

$$\mathcal{E}_\ell(c) = (c, b_\ell, m_\ell, n_\ell^c, n_\ell^{b_\ell}, \widehat{\Delta}_\ell(c), \text{CI}_\ell(c), v_\ell(c)), \quad (6)$$

where $\mathcal{E}_\ell(c)$ is the evidence record produced for candidate c at stage ℓ , which packages the comparison and its statistical summary into a single object. Within it, b_ℓ is the stage-specific baseline and m_ℓ is the evaluation signal. The quantities n_ℓ^c and $n_\ell^{b_\ell}$ denote the numbers of valid evaluation units for the candidate and baseline. $\widehat{\Delta}_\ell(c)$ denotes the estimated candidate effect, $\text{CI}_\ell(c)$ its confidence interval, and $v_\ell(c)$ the promotion decision. L1 and L2 use model-based proxy scores. L3 and L4 use observed online outcomes. This record is the unit of evidence accumulated in the research state and interpreted together with its experimental context.

Let $\widehat{\mu}_\ell(c)$ and $\widehat{\mu}_\ell(b_\ell)$ denote the estimated values of the evaluation signal for the candidate and baseline. With the signal oriented so that larger values indicate better outcomes, the estimated effect is

$$\widehat{\Delta}_\ell(c) = \widehat{\mu}_\ell(c) - \widehat{\mu}_\ell(b_\ell). \quad (7)$$

The confidence interval is centered on this effect and widened by its standard error,

$$\text{CI}_\ell(c) = \left[\widehat{\Delta}_\ell(c) - t_{v_\ell, 1-\alpha/2} \text{SE}_\ell(c), \widehat{\Delta}_\ell(c) + t_{v_\ell, 1-\alpha/2} \text{SE}_\ell(c) \right], \quad (8)$$

where $\text{SE}_\ell(c)$ is the standard error of the effect, $t_{v_\ell, 1-\alpha/2}$ is the critical value at significance level α , and v_ℓ the associated degrees of freedom. The evidence representation and promotion rule are shared across levels, whereas effect estimation follows the sampling structure of each evaluation protocol. The shared representation retains stage-specific signal definitions and confidence intervals rather than combining heterogeneous results into a single score. The stage-specific effect estimators and confidence computations are detailed in Appendix A.

Let $\text{CI}_\ell(c) = [L_\ell(c), U_\ell(c)]$ denote the confidence interval at the prescribed confidence level. The promotion decision is

$$v_\ell(c) = \begin{cases} \text{PROMOTE}, & L_\ell(c) > 0, \\ \text{RETAIN}, & L_\ell(c) \leq 0 \leq U_\ell(c), \\ \text{STOP}, & U_\ell(c) < 0. \end{cases} \quad (9)$$

PROMOTE indicates statistically significant evidence of a positive effect and advances the candidate to the next evaluation stage. At L4, it indicates that the candidate satisfies the promotion rule for deployment consideration. RETAIN indicates inconclusive evidence and retains the candidate for further evaluation at the current stage. STOP indicates statistically significant evidence of a negative effect and terminates the candidate's progression through the ladder.

The promotion decision progressively narrows the candidate set along the active evaluation sequence. For strategy task k , let $\mathcal{L}_k = (\ell_{k,1}, \dots, \ell_{k,J_k})$ denote its ordered sequence of active levels, where $\ell_{k,j} \in \{1, 2, 3, 4\}$

and $\ell_{k,j} < \ell_{k,j+1}$. The candidates entering the first active level are those generated by Evidence-driven AutoResearch, $\mathcal{A}_{k,t}^{[1]} = \mathcal{A}_{k,t}$, and each active level advances only the candidates it promotes:

$$\mathcal{A}_{k,t}^{[j+1]} = \{c \in \mathcal{A}_{k,t}^{[j]} : v_{\ell_{k,j}}(c) = \text{PROMOTE}\}, \quad j \in \{1, \dots, J_k\}, \quad (10)$$

which yields the nested selection $\mathcal{A}_{k,t}^{[1]} \supseteq \mathcal{A}_{k,t}^{[2]} \supseteq \dots \supseteq \mathcal{A}_{k,t}^{[J_k+1]}$. A candidate reaches an active level only after being promoted at every preceding level in the task-specific sequence. For deployment consideration, the sequence terminates at Decision-Grade Online Evaluation (L4), and $\mathcal{A}_{k,t}^{[J_k+1]}$ contains the candidates promoted at that level. Thus, lower-cost active levels screen candidates before costly online evaluation.

Evaluation results and promotion decisions are returned to the corresponding research state together with their experimental contexts. For strategy task k in experimental round t , $\mathcal{E}_{k,t}^{(\ell)}$ collects the records of candidates evaluated at stage ℓ . These records support evidence interpretation across experiments while preserving the conditions under which each result was obtained.

5.2.2 Model-Based Proxy Evaluation

L1 and L2 evaluate candidate ranking outputs without exposing users to those outputs. A list-level evaluation model provides a proxy score for each candidate. Given request context x and the ranked list $L^g(x)$ produced by strategy g , the proxy evaluation signal is

$$m_\ell(g; x) = R_\psi(x, L^g(x)), \quad \ell \in \{1, 2\}, \quad (11)$$

where R_ψ estimates the online objective from the request context and the complete ranked list. The model accounts for the composition and relative positions of multiple content formats. It therefore evaluates the combined ranking output rather than individual content scores in isolation.

The resulting proxy scores provide the evaluation signal for estimating candidate effects at L1 and L2. Both stages use the same scoring model but evaluate candidates under different request and execution conditions. Offline Replay uses fixed historical requests. Shadow-Traffic Evaluation uses duplicated real-time requests in the online execution environment.

5.2.3 L1: Offline Replay

L1 evaluates candidate modifications on a fixed set of historical requests. The candidate and baseline ranking procedures process the recorded request contexts and produce their respective ranked lists. The evaluation model assigns a proxy score to each list. For strategy $g \in \{c, b_1\}$, the estimated proxy score is

$$\hat{\mu}_1(g) = \widehat{\mathbb{E}}_{x \sim \mathcal{X}_{\text{hist}}} [m_1(g; x)], \quad (12)$$

where $\mathcal{X}_{\text{hist}}$ is the historical request set over which the proxy signal is averaged. Because the candidate and baseline process the same historical requests, L1 estimates the candidate effect from request-level paired differences.

The verifier forms the effect $\hat{\Delta}_1(c)$ and its confidence interval $\text{CI}_1(c)$ from these scores under the Offline Replay protocol. This evidence supports low-cost candidate screening. The promoted subset $\mathcal{A}_{k,t}^{(2)}$ proceeds to Shadow-Traffic Evaluation.

5.2.4 L2: Shadow-Traffic Evaluation

L2 evaluates candidate modifications using duplicated real-time requests in the industrial search system. Candidate ranking procedures execute with current request contexts and online features. Their outputs are used for evaluation without replacing the results shown to users. The per-request proxy signal $m_2(g; x)$ follows the same model-based definition, with $L^g(x)$ produced in the online execution environment. The scoring model is shared with L1, but its inputs reflect current requests, online features, and the ranking outputs generated under these conditions. For strategy $g \in \{c, b_2\}$, the estimated proxy score is

$$\hat{\mu}_2(g) = \widehat{\mathbb{E}}_{x \sim \mathcal{X}_{\text{shadow}}^g} [m_2(g; x)], \quad (13)$$

where $\mathcal{X}_{\text{shadow}}^g$ is the set of requests received by the shadow-traffic group for strategy g . Candidate and baseline statistics are computed from independent shadow-traffic groups that receive requests replicated from the live traffic stream, without request-level pairing across groups.

The verifier forms the effect $\hat{\Delta}_2(c)$ and its confidence interval $\text{CI}_2(c)$ from these scores, which determine candidate promotion under the unified policy. This stage examines candidate performance under current operating conditions while retaining model-based proxy evaluation. The promoted subset $\mathcal{A}_{k,t}^{(3)}$ proceeds to Rapid Online Evaluation.

5.2.5 L3: Rapid Online Evaluation

L3 applies candidate modifications to limited randomized traffic. Users in the candidate group receive results produced by the candidate strategy, and users in the baseline group receive results produced by the baseline strategy. The experiment collects observed user outcomes over a short observation window. These outcomes replace model-based proxy scores as the evaluation signal.

Let $\mathcal{N}_3^g$ denote the evaluation units observed under strategy g , and let $y_3(g; u)$ denote the observed objective signal for unit u . For strategy $g \in \{c, b_3\}$, the estimated value is

$$\hat{\mu}_3(g) = \widehat{\mathbb{E}}_{u \sim \mathcal{N}_3^g} [y_3(g; u)], \quad (14)$$

where the expectation is taken over the units randomly assigned to each strategy. Evaluation units and outcome definitions follow the online experimental protocol. The verifier forms the effect $\hat{\Delta}_3(c)$ and its confidence interval $\text{CI}_3(c)$ under that protocol and applies the unified promotion policy. This stage assesses whether candidates selected through proxy evaluation produce positive effects in real-user outcomes. The promoted subset $\mathcal{A}_{k,t}^{(4)}$ proceeds to Decision-Grade Online Evaluation.

5.2.6 L4: Decision-Grade Online Evaluation

L4 evaluates promoted candidates through online experiments with broader randomized traffic and a longer observation window. Candidate and baseline groups provide observed outcomes aligned with the optimization objective. The expanded experiment covers a wider range of traffic conditions and temporal variation.

Using the same notation for observed objective signals, the estimated value is

$$\hat{\mu}_4(g) = \widehat{\mathbb{E}}_{u \sim \mathcal{N}_4^g} [y_4(g; u)], \quad g \in \{c, b_4\}, \quad (15)$$

where $\mathcal{N}_4^g$ contains the evaluation units used in Decision-Grade Online Evaluation and $y_4(g; u)$ is the objective signal measured over its observation window. The resulting effect $\hat{\Delta}_4(c)$ and its confidence interval $\text{CI}_4(c)$ support the final deployment decision. Candidates satisfying the promotion rule at L4 are eligible for deployment consideration. Evaluation results and final decisions update the corresponding research states and inform subsequent search.

6 Experiments

6.1 Experimental Setup and Research Questions

The experiments evaluate PEAR in a real-world industrial search system. Three anonymized strategy tasks provide records of candidate exploration and evaluation. These records include experimental configurations, evaluation results, and hypothesis revisions. A/B results from Decision-Grade Online Evaluation are reported for strategies identified for two of these tasks.

The evaluation addresses three research questions:

RQ1: Evidence-driven hypothesis revision. How are accumulated evaluation results incorporated into hypothesis revision and subsequent candidate generation?

RQ2: Progressive candidate evaluation. How do proxy evaluation results relate to observed online outcomes, and how do selected candidates behave across evaluation stages?

RQ3: Online effectiveness. Do strategies optimized with PEAR improve the online optimization objective during Decision-Grade Online Evaluation?

Implementation Detail. Experiments at L1–L3 run on timescales of hours, and L4 experiments run on timescales of days. Main Order/DAU is the primary outcome reported for the final online A/B experiments; the remaining reported outcomes are auxiliary business metrics for the e-commerce search setting. Traffic conditions, observation windows, and evaluation signals differ across stages. Cross-stage analysis therefore examines the direction of candidate effects rather than directly comparing their magnitudes. For industrial search optimization, each strategy task instantiates an ordered sequence of active evaluation levels according to data availability and validation requirements. Candidates must satisfy the promotion criterion at each active level before advancing to the next level in that sequence. In the following experiments, data-access restrictions across regions preclude the use of Offline Replay (L1), so the evaluated sequence begins at Shadow-Traffic Evaluation (L2) and the subsequent experimental sections do not involve L1 validation. Although L1 is not used in these experiments, it has assisted algorithm engineers in optimizing other strategies in production.

6.2 RQ1: Evidence-Driven Hypothesis Revision

Table 1 L2 search scale and candidate selection. Unique configurations are deduplicated within each phase; promotion rate is the number of candidates promoted from L2 divided by the number of experimental groups.

Task phase	Experimental rounds	Groups	Unique configs.	Promoted from L2	Promotion rate
A	19	133	20	2	1.50%
B–I	12	84	23	5	5.95%
B–II	2	14	6	1	7.14%
B–III	21	147	53	2	1.36%
C	16	76	54	3	3.95%

Candidate exploration and selection. Table 1 summarizes candidate exploration at L2 for three anonymized strategy tasks. Each task evaluates multiple configurations and retains a small subset for online evaluation. Task B comprises three consecutive search phases: the first two explore local parameter neighborhoods, and Phase III expands the candidate range based on accumulated evaluation results. The following trajectories examine how this evidence informs hypothesis revision and subsequent candidate generation.

Table 2 Representative hypothesis-revision trajectory for Task A. The table shows four early experimental rounds from a 19-round search; $G = (w, q)$ denotes the global window and capacity constraint.

Experimental round / research question	Controlled interventions	Evidence-driven update
1. Which constraint axis accounts for the observed gain?	Apply one-axis changes to global and format-specific constraints; also test the coupled global change $G = (3, 2)$.	Only $G = (3, 2)$ yields a significant positive order signal. The effects of increasing w , increasing q , and experimental variation remain entangled.
2. Can $G = (3, 2)$ be reproduced, and which variable explains its gain?	Replicate $G = (3, 2)$; compare window-only $G = (3, 1)$, capacity-only $G = (2, 2)$, and boundary settings including $G = (3, 3)$.	$G = (3, 3)$ produces a stronger signal. Replications of $G = (3, 2)$ yield mixed directional effects and indicate potential degradation in the exit metric. The next experimental round prioritizes capacity and replication.
3. Is the gain driven by the coupled change or by capacity alone?	Replicate $G = (3, 3)$ and $G = (3, 2)$; test capacity-only $G = (2, 2)$ and wider-window boundary settings.	$G = (2, 2)$ achieves the strongest observed result. The revised hypothesis attributes the gain primarily to capacity relaxation without requiring a wider window.
4. Are the marginal effects of window and capacity stable?	Concentrate replications on $G = (2, 2)$, $G = (3, 2)$, and $G = (3, 3)$; retain window-only $G = (3, 1)$ as a counterfactual.	$G = (3, 3)$ remains directionally favorable on order and exit signals but is not significant. The capacity-centered hypothesis is retained for further replication.

Hypothesis revision through controlled interventions. Task A illustrates how controlled interventions distinguish competing explanations for an observed gain. The task optimizes constraints on e-commerce result density. Let $G = (w, q)$ denote the global constraint, where w is the window size and q is the maximum content capacity within that window; the deployed baseline is $G = (2, 1)$. Table 2 traces four representative early experimental rounds. Other constraint families remain at their baseline values after initial screening.

The initial gain from $G = (3, 2)$ leaves the effects of window size and capacity unresolved. The next experimental round separates these factors and replicates the coupled setting. Mixed directional effects across replications motivate further tests of capacity relaxation. In Experimental Round 3, the capacity-only setting $G = (2, 2)$ achieves the strongest observed result, shifting the hypothesis toward capacity relaxation rather than window expansion. Experimental Round 4 then examines the consistency of this hypothesis through additional controlled comparisons and replication.

Hypothesis revision from online evaluation. Task B optimizes position-dependent exposure settings for e-commerce search results. Table 3 summarizes three search phases and their outcomes from Decision-Grade Online Evaluation.

Table 3 Outcomes from Decision-Grade Online Evaluation across search phases for Task B.

Phase	Experimental rounds	Groups	Unique configs.	Promoted from L2	L4 outcome
I	12	84	23	5	No significant gain
II	2	14	6	1	Positive, not significant
III	21	147	53	2	Significant order gain

In the first two phases, candidates promoted from L2 do not achieve significant improvements at L4. These online evaluation results are incorporated into the research state to reassess the hypotheses underlying candidate generation. The revised hypotheses guide further exploration at L2, expanding the candidate

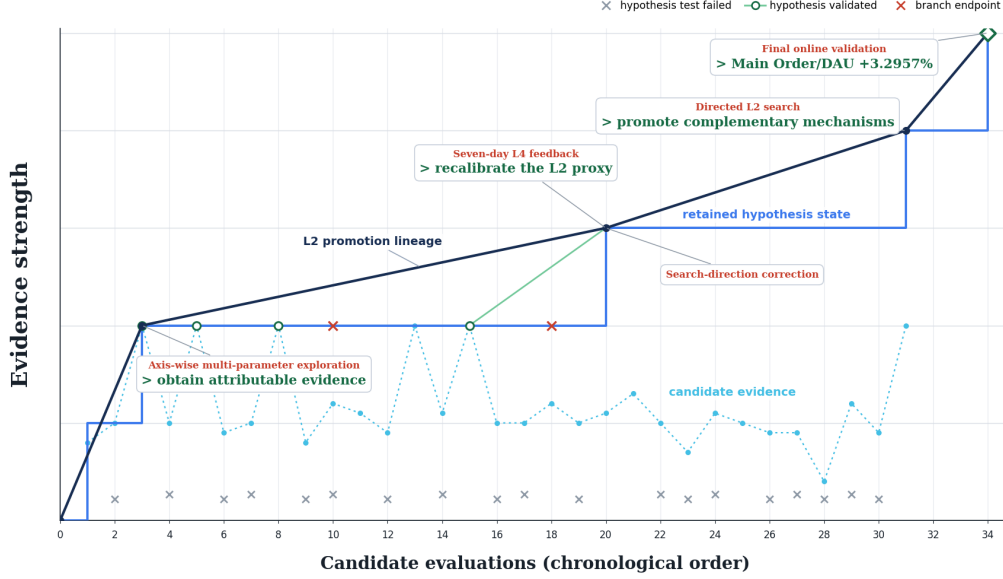


Figure 2 Hypothesis-driven search trajectory for Task B. Candidate evaluations update a retained hypothesis state through validation, rejection, and branch termination until Decision-Grade Online Evaluation.

range beyond the initial parameter neighborhoods. Figure 2 illustrates this exploration process: the agent uses accumulated evaluation evidence to revise hypotheses, terminate unsupported branches, and redirect candidate generation. In Phase III, the revised exploration identifies candidates for further online evaluation, one of which subsequently achieves a significant improvement at L4. The trajectory illustrates how evidence from Decision-Grade Online Evaluation is incorporated into hypothesis revision and subsequent candidate generation, connecting candidate promotion with continued search.

6.3 RQ2: Progressive Candidate Evaluation

Aggregate agreement between proxy predictions and online outcomes. An independent controlled experiment compares mean predicted and observed click, order, and GMV outcomes at a common session granularity. Table 4 reports the signed relative errors between these aggregate means. The order component has the smallest absolute relative error, with a signed value of -3.49% . This comparison assesses aggregate agreement rather than the accuracy of predicted effects for individual candidate interventions.

Table 4 Signed relative errors between the aggregate means of proxy predictions and observed online outcomes.

	Click	Order	GMV
Signed relative error	-16.02%	-3.49%	-18.23%

Candidate effects across evaluation stages. Table 5 follows two Task A candidates from L2 through L3 to L4. Both traced candidates have positive order proxy effects at L2 and retain positive observed effects at L3 and L4. These cases illustrate directional consistency from proxy-based screening to Decision-Grade Online Evaluation.

Together, the aggregate comparison and the two candidate trajectories provide illustrative evidence that proxy evaluation can support candidate screening, while online evaluation remains necessary to establish effects under real-user traffic.

Table 5 Candidate effects across evaluation stages for two Task A candidates. L2 reports changes in the order proxy, L3 reports changes in observed orders, and L4 reports changes in Main Order/DAU.

Candidate	L2: Shadow-Traffic Evaluation	L3: Rapid Online Evaluation	L4: Decision-Grade Online Evaluation
A	Order proxy +45.88%	Observed order +15.48%	Main Order/DAU +2.73%
B	Order proxy +61.64%	Observed order +29.76%	Main Order/DAU +2.16%

6.4 RQ3: Online Effectiveness

Table 6 reports Decision-Grade Online Evaluation results for strategies optimized with PEAR for two search optimization tasks. Relative to their respective baselines, the strategies for Task A and Task B significantly improve Main Order/DAU by 2.7336% and 3.2957%, respectively. Both strategies also achieve significant gains in ASN, SKU Order/DAU, Main OPMS, and SKU OPMS.

Table 6 Relative metric changes in online A/B evaluations of strategies optimized with PEAR for two search optimization tasks. All values are relative to the corresponding baseline; dark-green boldface indicates statistical significance at $p < 0.05$.

Task	SearchPV/DAU	ASN	GMV/DAU	Main Order/DAU	SKU Order/DAU	PayPV/PV	Main OPMS	SKU OPMS
A	+0.0063%	+1.1512%	+2.2136%	+2.7336%	+2.7259%	+1.0441%	+2.7407%	+2.7375%
B	+0.0470%	+1.3841%	+1.6633%	+3.2957%	+2.9826%	+1.6298%	+3.2595%	+2.9448%

7 Conclusion

PEAR is an AutoResearch framework for industrial search optimization under non-stationary outcomes and heterogeneous evaluation signals. Evidence-driven AutoResearch associates hypotheses with evidence and its experimental context to support reliable search knowledge accumulation. Confidence-Gated Verifier Ladder connects four evaluation levels through a unified confidence-based promotion gate. The two components jointly turn evaluation results into signals that guide both candidate promotion and hypothesis revision, linking lower-cost exploration with Decision-Grade Online Evaluation.

Experiments in a real-world industrial search system show how accumulated evidence informs controlled interventions and how online evaluation results redirect subsequent search. Strategies optimized with PEAR increased Main Order/DAU by 2.7336% and 3.2957% relative to their respective baselines in two A/B experiments at the Decision-Grade Online Evaluation stage. These results support the use of contextual evidence and progressive evaluation for iterative industrial search optimization. Future work will examine broader intervention scopes, including model architectures and training procedures.

8 Limitations

The current work is confined to a single e-commerce general search environment, and PEAR’s generality across other search platforms, geographic regions, and evaluation configurations remains to be established. Where compliance constraints preclude Offline Replay (L1), evaluation proceeds from L2 onward while retaining the higher-fidelity validation stages. The search space is currently restricted to strategy-level parameters; its extension to model parameters, architectures, and training procedures is left to future work. Confidence-Gated Verifier Ladder also depends on the compute clusters, deployed evaluation and search models, and live traffic of the underlying search environment, which constrains its applicability in settings where such resources are unavailable.

9 Ethical Considerations

In the evaluated deployment, AutoResearch is employed to optimize an e-commerce general search system. As optimization feedback, the agents receive model-based proxy scores produced by the list-level evaluation model and aggregate effect estimates of the objective signal from online experiments, rather than user-level records, personal identifiers, sensitive attributes, or other sensitive information. All data used for evaluation and experimentation are processed under applicable data-governance and compliance requirements. The optimization loop therefore operates on compliant evaluation signals within the existing authorized experimentation pipeline and does not require additional access to personal or sensitive data.

Before any candidate is exposed to real-user traffic, the proposed change and supporting evidence undergo human review and require explicit human confirmation; automated promotion within the Verifier Ladder cannot independently initiate such exposure.

10 Acknowledgments

We thank our colleagues on the E-commerce General Search, Architecture, Product, and Data Science teams for their invaluable contributions to the design, development, and deployment of the systems described in this work. We are grateful to the AutoSearch tooling team for building the compute cluster and infrastructure that enabled large-scale AutoResearch for industrial search system optimization.

Author List

Global E-Commerce Agentic Search Team: Yifan Wang^{*}, Shipeng Zhu, Fei Xiong, Yuqin Yang, Yonghui Huang, Kunyao Wu, Yue Wang, Weichao Meng, Yu Gong[†].

^{*}: First author, yifanwang993w@gmail.com

[†]: Corresponding author, gy910210@gmail.com

References

- [1] Boiko, D. A., MacKnight, R., Kline, B., and Gomes, G. (2023). Autonomous chemical research with large language models. *Nature*, 624(7992):570–578.
- [2] Chen, G., Chen, J., Chen, L., Zhao, J., Meng, F., Zhao, W. X., Song, R., Chen, C., Wen, J.-R., and Jia, K. (2026). Toward autonomous long-horizon engineering for ml research. *arXiv preprint arXiv:2604.13018*.
- [3] Chen, Y., Liu, Q., Zhang, Y., Sun, W., Ma, X., Yang, W., Shi, D., Mao, J., and Yin, D. (2025). Tourrank: Utilizing large language models for documents ranking with a tournament-inspired strategy. In *Proceedings of the ACM on Web Conference 2025*, WWW ’25, page 1638–1652, New York, NY, USA. Association for Computing Machinery.
- [4] Cheng, Y., Zhou, L., Liang, X., Luo, D., Lee, T., Zheng, K., Zhang, W., Cai, M., Dong, J., and Zhang, A. (2026). Let the agent steer: Closed-loop ranking optimization via influence exchange. *arXiv preprint arXiv:2603.27765*.
- [5] Huang, Q., Vora, J., Liang, P., and Leskovec, J. (2024). MLAGentBench: Evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*.
- [6] Kandasamy, K., Dasarathy, G., Schneider, J., and Póczos, B. (2017). Multi-fidelity bayesian optimisation with continuous approximations. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1799–1808. PMLR.
- [7] Kim, S., Park, S., Kang, H., Kim, W., Seo, J., In, Y., Yoon, K., Jeon, H., and Park, C. (2026). Self-evolverec: Self-evolving recommender systems with llm-based directional feedback. *arXiv preprint arXiv:2602.12612*.

- [8] Kohavi, R., Longbotham, R., Sommerfield, D., and Henne, R. M. (2009). Controlled experiments on the web: Survey and practical guide. *Data Mining and Knowledge Discovery*, 18:140–181.
- [9] Lao, C., Pan, F., Ma, G., Li, H., Lin, H., Shi, J., Zhao, K., Gai, K., Zhou, M., Zhou, Q., et al. (2026). Agentx: Towards agent-driven self-iteration of industrial recommender systems. *arXiv preprint arXiv:2606.26859*.
- [10] Lee, Y., Nair, R., Zhang, Q., Lee, K., Khattab, O., and Finn, C. (2026). Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.
- [11] Li, L., Chu, W., Langford, J., and Wang, X. (2011). Unbiased offline evaluation of contextual-bandit-based news article recommendation algorithms. In *Proceedings of the Fourth ACM International Conference on Web Search and Data Mining*, pages 297–306.
- [12] Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., and Talwalkar, A. (2018). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52.
- [13] Liu, J., Qiu, S., Li, M., Li, B., Ji, H., Han, S., Ye, X., Xia, P., Dong, Z., Chen, M., et al. (2026). Autoresearchclaw: Self-reinforcing autonomous research with human-ai collaboration. *arXiv preprint arXiv:2605.20025*.
- [14] Liu, Z., Li, C., Xiao, S., Li, C., Zhang, C. J., Liao, H., Lian, D., and Shao, Y. (2025). Fitting into any shape: A flexible llm-based re-ranker with configurable depth and width. In *Proceedings of the ACM on Web Conference 2025, WWW '25*, page 3942–3951, New York, NY, USA. Association for Computing Machinery.
- [15] Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., and Ha, D. (2024). The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.
- [16] Mann, T. A., Goyal, S., Gyorgy, A., Hu, H., Jiang, R., Lakshminarayanan, B., and Srinivasan, P. (2019). Learning from delayed outcomes via proxies with applications to recommender systems. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, pages 4324–4332. PMLR.
- [17] Park, J. S., O’Brien, J., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22.
- [18] Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., and Sun, M. (2024). ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15174–15186.
- [19] Romera-Paredes, B., Barekatin, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J. R., Ellenberg, J. S., Wang, P., Fawzi, O., Kohli, P., and Fawzi, A. (2024). Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475.
- [20] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36.
- [21] Tie, G., Shi, J., Song, D., Huang, Y., Sheng, Z., Zhou, X., Liu, D., Zhou, P., Chen, Y., Xu, R., et al. (2026). Autoresearch ai: Towards ai-powered research automation for scientific discovery. *arXiv preprint arXiv:2605.23204*.
- [22] Wang, H., Wu, Y., Chang, D., Wei, L., and Heldt, L. (2026). Self-evolving recommendation system: End-to-end autonomous model optimization with llm agents. *arXiv preprint arXiv:2602.10226*.
- [23] Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345.
- [24] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. (2023a). Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822.

- [25] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023b). React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.
- [26] Yoon, S., Kim, G., Cho, G.-H., and won Hwang, S. (2025). Acurank: Uncertainty-aware adaptive computation for listwise reranking.
- [27] Zhou, J., Zheng, Y., Chen, W., Zheng, Q., Su, H., Zhang, W., Meng, R., and Shen, X. (2025). Beyond content relevance: Evaluating instruction following in retrieval models.

A Confidence Computation for the Unified Promotion Policy

This appendix specifies the stage-specific estimators used by the unified promotion policy (Section 5.2) to obtain the candidate effect $\hat{\Delta}_\ell(c)$ and its confidence interval $\text{CI}_\ell(c)$. The evidence schema and promotion rule are shared across levels, while the estimator follows the sampling structure of each evaluation protocol. L1 uses request-aligned paired observations. L2 uses independent shadow-traffic groups, and L3 and L4 use independent randomized online groups.

Paired estimation for Offline Replay (L1). Let x_i denote the i -th historical request shared by candidate c and baseline b_1 , and define the request-level proxy-score difference as

$$d_i = m_1(c; x_i) - m_1(b_1; x_i), \quad i \in \{1, \dots, n\}. \quad (16)$$

The L1 effect, variance of the paired differences, standard error, and degrees of freedom are

$$\hat{\Delta}_1(c) = \bar{d} = \frac{1}{n} \sum_{i=1}^n d_i, \quad s_d^2 = \frac{1}{n-1} \sum_{i=1}^n (d_i - \bar{d})^2, \quad (17)$$

$$\text{SE}_1(c) = \frac{s_d}{\sqrt{n}}, \quad \nu_1 = n - 1. \quad (18)$$

This paired estimator controls for request-level variation because both strategies are evaluated on the same historical requests.

Independent-group estimation for L2–L4. For $\ell \in \{2, 3, 4\}$ and strategy $g \in \{c, b_\ell\}$, let $x_{g,i}^{(\ell)}$ denote its i -th valid evaluation unit and let $n_{g,\ell}$ be the number of such units. L2 units are collected from independent shadow-traffic groups without request-level pairing, whereas L3 and L4 units are collected from independently randomized online groups. The per-strategy mean and variance are

$$\bar{x}_{g,\ell} = \frac{1}{n_{g,\ell}} \sum_{i=1}^{n_{g,\ell}} x_{g,i}^{(\ell)}, \quad s_{g,\ell}^2 = \frac{1}{n_{g,\ell} - 1} \sum_{i=1}^{n_{g,\ell}} (x_{g,i}^{(\ell)} - \bar{x}_{g,\ell})^2. \quad (19)$$

The estimated effect and Welch standard error are

$$\hat{\Delta}_\ell(c) = \bar{x}_{c,\ell} - \bar{x}_{b_\ell,\ell}, \quad \text{SE}_\ell(c) = \sqrt{\frac{s_{c,\ell}^2}{n_{c,\ell}} + \frac{s_{b_\ell,\ell}^2}{n_{b_\ell,\ell}}}, \quad (20)$$

with Welch–Satterthwaite degrees of freedom

$$\nu_\ell = \frac{\left(\frac{s_{c,\ell}^2}{n_{c,\ell}} + \frac{s_{b_\ell,\ell}^2}{n_{b_\ell,\ell}} \right)^2}{\frac{\left(s_{c,\ell}^2 / n_{c,\ell} \right)^2}{n_{c,\ell} - 1} + \frac{\left(s_{b_\ell,\ell}^2 / n_{b_\ell,\ell} \right)^2}{n_{b_\ell,\ell} - 1}}. \quad (21)$$

Confidence interval. For every active level, the two-sided confidence interval at significance level α is

$$\text{CI}_\ell(c) = \left[\widehat{\Delta}_\ell(c) - t_{\nu_\ell, 1-\alpha/2} \text{SE}_\ell(c), \widehat{\Delta}_\ell(c) + t_{\nu_\ell, 1-\alpha/2} \text{SE}_\ell(c) \right]. \quad (22)$$

When a relative effect is reported, the effect and interval endpoints are normalized by the corresponding baseline mean. By default, $\alpha = 0.05$, corresponding to 95% confidence intervals.

Promotion decision. With the evaluation signal oriented so that larger values indicate better outcomes, the promotion decision applies the interval endpoints $L_\ell(c)$ and $U_\ell(c)$ as in Section 5.2. A candidate is promoted when the interval lies entirely above zero ($L_\ell(c) > 0$), which provides statistically significant evidence of a positive effect at the current significance level. It is retained for further evaluation when the interval contains zero ($L_\ell(c) \leq 0 \leq U_\ell(c)$), which indicates that the observed effect is not yet distinguishable from no effect. It is stopped when the interval lies entirely below zero ($U_\ell(c) < 0$), which provides statistically significant evidence of a negative effect. The relative interval CI_c^{rel} shares the sign of the absolute interval and yields the same decision; it is reported for interpretability.

B Prompt Templates and Structured Outputs for Research-State Updates

The following templates summarize the instructions for hypothesis revision and cross-round evidence reuse in Evidence-driven AutoResearch. They retain the core inputs, update rules, and outputs while omitting deployment-specific identifiers and serialization details.

B.1 Hypothesis Update Prompt

Prompt Template for Hypothesis Update

Objective and inputs. Maintain testable parameter regularities across experimental rounds. Read the completed round summary, candidate records, confidence evidence, final review, lessons, and existing hypotheses.

Update instructions.

1. Generate at most three new hypotheses per round. Specify the parameter range, expected metric direction, risk boundary, and implications for candidate design; do not merely restate the best candidate.
2. Merge equivalent claims with overlapping scopes. Record narrower findings as scoped evidence without broadening their support. Keep claims with different directions or risk conditions separate and link them through **Related**.
3. Assign **support** when covered evidence supports the claim under the evaluation criteria, **refute** when it contradicts the claim or violates a decision constraint, and **neutral** when scope coverage or evidence is insufficient. Each round contributes at most one verdict per hypothesis.
4. Increment the corresponding counter and set **Score** = **Support** - **Refute**. Scores of at least 2 support **active** status; scores of at most -2 indicate **risky** hypotheses. Archive superseded or persistently unused hypotheses.

Plan use. Retrieve relevant active hypotheses with scores of at least 2 and risky hypotheses with scores of at most -2. Require a matching strategy version and explain each hypothesis’s effect on candidate design. If none applies, mark the round as exploratory. These scores summarize hypothesis evidence; they are not verifier confidence estimates.

Hypothesis Output Template.

```
### H-{id}: {testable parameter regularity}
- **Status:** active / risky / archived
- **Strategy Version:** {strategy fingerprint}
- **Score:** {Support - Refute}
- **Support:** {supporting-round count}
```

- ****Refute:**** {refuting-round count}
- ****Neutral:**** {neutral-round count}
- ****Scope:**** {parameter ranges and experimental conditions}
- ****Claim:**** {expected metric direction under these conditions}
- ****Plan Use:**** {implications for subsequent candidate design}
- ****Risk:**** {trade-offs, applicability limits, or counterexamples}
- ****Evidence:**** {round:candidate verdict; ...}
- ****Related:**** {related hypothesis identifiers or none}
- ****Updated At:**** {timestamp}

Plan-Stage Output Template.

Used hypotheses: H-{id} score={value}: {design implication}

Candidate design: {candidate}: {modification; hypothesis tested}

B.2 Cross-Round Memory Update Prompt

Prompt Template for Cross-Round Memory Update

Objective and inputs. Maintain two complementary records: structured lessons preserve round-level facts and reasoning; compressed memory summarizes cross-round findings and guides subsequent candidate design. Use all round summaries for the current strategy and the previous memory.

Update instructions.

1. After the final review, append one non-failure lesson per round with an Outcome matching the recorded decision. Record execution failures separately as crash lessons with their causes.
2. Preserve the strategy version, experimental context, decision rationale, and evidence references. When summaries disagree with measured evidence or experimental records, correct the summaries.
3. Before planning, retrieve relevant lessons with a matching strategy version. Use supported findings, avoid repeatedly unsuccessful directions, and resolve recurring execution failures. Incompatible lessons remain available for audit, not direct candidate guidance.
4. Rewrite compressed memory after each round using the current strategy's summaries and preceding memory. Preserve current-run lessons, down-weight older findings, and compress related historical lessons without losing evidence references. Provide concrete guidance for subsequent candidate design.

Outcome semantics. keep, discard, and iterate summarize the recorded round decision; pivot denotes a change of strategy family, crash an execution failure, and summary a historical synthesis.

Lesson Output Template.

L-{N}: {title}

- ****Strategy:**** {candidate settings and search design}
- ****Strategy Version:**** {strategy fingerprint}
- ****Outcome:**** keep / discard / iterate / pivot / crash / summary
- ****Insight:**** {actionable finding to reuse or avoid}
- ****Context:**** goal=...; scope=...; metric=...; direction=...
- ****Iteration:**** {round identifier}#{task identifier}
- ****Reason:**** {rationale consistent with the recorded decision}
- ****Refs:**** trace={record}; experiment={id}; round={id}
- ****Timestamp:**** {timestamp}

Compressed Memory. The source protocol specifies its content rather than a fixed output schema: reusable findings, their applicability conditions and evidence references, and concrete candidate-design guidance for the next round.

B.3 Structured Experimental Records

Structured Output Templates for Experimental Records

Recording instructions. Link candidate designs, confidence evidence, and the final review through shared task and experimental-round identifiers. Retain one baseline per round and record each candidate's parameters and design rationale. Base selection and the final decision on the confidence-evaluation record, not legacy score logs. The templates below retain selected source fields; placeholders replace deployment-specific values.

State and decision. The source protocol uses `cycle` for an experimental round. Its state is `RUNNING`, `CLOSED`, or `BLOCKED`. Completed rounds record `KEEP`, `DISCARD`, or `ITERATE`, mapped to the corresponding lesson outcomes. Blocked rounds record a failure cause without a decision. These round-level labels are distinct from the verifier promotion labels in the Method.

Final Review Template.

```
{
  "task_name": "<task identifier>",
  "cycle_id": "<round identifier>",
  "stage": "review",
  "status": "DONE",
  "decision": "<KEEP | DISCARD | ITERATE>",
  "reason": "<evidence-based decision rationale>",
  "best_version": "<selected candidate identifier>",
  "best_selected_by": "llm_from_confidence_pipeline",
  "confidence_method": "<evaluation method>",
  "confidence_file": "<confidence-evidence record>",
  "best_confidence": {},
  "strategy_contract_version": "<strategy fingerprint>",
  "versions_file": "<candidate-design records>"
}
```

Evidence Trace Template.

```
{
  "task_name": "<task identifier>",
  "round": "<round label>",
  "cycle": {
    "cycle_id": "<round identifier>",
    "status": "CLOSED",
    "best_version": "<selected candidate identifier>",
    "confidence_method": "<evaluation method>",
    "confidence_file": "<confidence-evidence record>",
    "best_confidence": {},
    "decision": "<KEEP | DISCARD | ITERATE>",
    "versions_file": "<candidate-design records>"
  },
  "decision": "<same round decision>"
}
```

The `best_confidence` object contains the selected candidate's confidence evidence. Candidate, decision, and evidence references must agree across the review, trace, and lesson records.

B.4 Knowledge-Augmented Candidate Generation Prompt

This template summarizes the Bayesian-optimization-inspired reasoning principles $\mathcal{K}_{\text{BO}}$ used in Equation 4. These principles guide qualitative reasoning about observed effects, uncertainty, and candidate diversity.

Prompt Template for Knowledge-Augmented Candidate Generation

Inputs. Mutable parameters and their types, intervention bounds, baseline, candidate budget, historical evaluations, hypotheses, memory, and exact-replication requirements.

Generation instructions.

1. Classify the parameter space and diagnose the search state as `cold_start`, `local_refine`, `stagnant`, or `pivot`. Use repeated evidence, coverage, and contradictions rather than the largest point estimate alone.
2. Select one reasoning strategy. Use **GP-inspired** reasoning for low-dimensional continuous spaces: contrast supported regions with uncertainty gaps. Use **TPE-inspired** reasoning for mixed continuous and integer spaces: distinguish supported, unfavorable, and unresolved parameter patterns. Use **SMAC-inspired** reasoning for categorical or conditional spaces: compare supported branches and structural counterfactuals.
3. Form a candidate pool at least twice the available new-candidate budget. Cover refinement, boundary extension, uncertainty probes, and interaction tests. Preserve negative and inconclusive evidence when selecting candidates.
4. Under stagnation or a change of search direction, increase exploration. Include a feasible point beyond the explored range on a meaningful axis and an interaction test or uncovered branch. Remain within the intervention bounds.
5. Remove invalid, excluded, or duplicate proposals and select a diverse subset. Keep exact replications separate from new candidates: replication preserves the full configuration, while every new configuration is generated through the selected reasoning strategy.

Output Template. Report concise, evidence-backed selection summaries and candidate roles. The following layout summarizes the required fields without reproducing the full planning schema.

```
ideation_strategy.method: <GP-/TPE-/SMAC-inspired>
search_state: <cold_start | local_refine | stagnant | pivot>
cycle_mode: <current experiment-allocation mode>
exploration_level: <level; high for stagnant or pivot>
selection_reason: <brief evidence-backed rationale>
Allocation: <exact-confirmation and new-candidate counts>
For each candidate:
  candidate_source: <confirmation_queue | algorithm>
  search_role: <replication or method-specific exploration role>
```